

```
public:
    virtual void init()=0;
};

#endif
```

Let's start by making a sample shared library, *testLib.h*:

```
#ifndef TESTLIB_H
#define TESTLIB_H

class TestLib
{
public:
    void init();
};

#endif
```

In the above header file, we declared a class and the *init* method as public, as we need to access it later. The next code is *testLib.cpp*:

```
#include <iostream>
#include "testVir.h"
#include "testLib.h"

using namespace std;
void TestLib::init()
{
    cout<<"TestLib::init: Hello World!! "<<endl ;
}

//Define functions with C symbols (create/destroy TestLib
instance).
extern "C" TestLib* create()
{
    return new TestLib;
}

extern "C" void destroy(TestLib* Tl)
{
    delete Tl ;
}
```

Our two-class helper functions with *extern "C"* and will be used to create and destroy an instance of the *shared* library class, and serve as access points (entry and exit points) of the library. Let us compile and link the above class with the *shared* and *fPIC* options to *g++*:

```
g++ -shared -fPIC testLib.cpp -o testLib.so
```

Now, let's write a program to access this library—*main.cpp*:

```
#include<iostream>
#include<dlfcn.h>
#include "testVir.h"

using namespace std;

int main()
{
    void *handle;
    handle = dlopen("./testLib.so", RTLD_NOW);
    if (!handle)
    {
        printf("The error is %s", dlerror());
    }

    typedef TestVir* create_t();
    typedef void destroy_t(TestVir*);

    create_t* creat=(create_t*)dlsym(handle,"create");
    destroy_t* destroy=(destroy_t*)dlsym(handle,"destroy");
    if (!creat)
    {
        cout<<"The error is %s"<<dlerror();
    }
    if (!destroy)
    {
        cout<<"The error is %s"<<dlerror();
    }
    TestVir* tst = creat();
    tst->init();
    destroy(tst);
    return 0 ;
}
```

In the above program, we used *dlopen* to load the library, then retrieved references to the symbols for our two access functions with *dlsym*, and then via these, first invoked the *create* function to get an instance of the shared object, invoked its *init* method, and then destroyed the object used to invoke the methods of the C++ class. Next, compile and link the program with *g++ -ldl main.cpp -o test*. The option *-ldl* is used to link it with *libdl.so* to use the methods in the *dlfcn* header file.

The above scenario is basically used to build a plug-in framework, where the plug-ins are dynamically loaded into the main component. 

By: Prashant Jadhav

The author works as a software engineer at Amdocs, Pune. He is an open-source enthusiast, and likes to try new things and hack in his spare time. He promotes open and free software, and does programming in the open-source domain.